

EJERCICIOS SEGUIMIENTO QUINTA ACTIVIDAD

ESTUDIANTE

Juan Jose Velez Bustamante juvelezb@unal.edu.co

DOCENTE

Walter Hugo Arboleda



Universidad Nacional de Colombia

Facultad de Minas

Medellin 06/07/2026

Actividad 5 — Aplicación de Interfaz Gráfica con Archivo

Programación Orientada a Objetos · Formulario CRUD (Create, Read, Update, Delete)

1. Descripción del proyecto

Se desarrolló una aplicación de escritorio en Python con interfaz gráfica (Tkinter) que gestiona una lista de contactos persistida en un archivo de texto plano. La aplicación implementa las cuatro operaciones fundamentales de manejo de archivos: **Create** (crear un contacto nuevo), **Read** (leer/listar los contactos existentes), **Update** (actualizar un contacto seleccionado) y **Delete** (eliminar un contacto). El diseño separa claramente el modelo de datos, la capa de persistencia y la interfaz gráfica, siguiendo los principios de la Programación Orientada a Objetos (encapsulamiento y responsabilidad única).

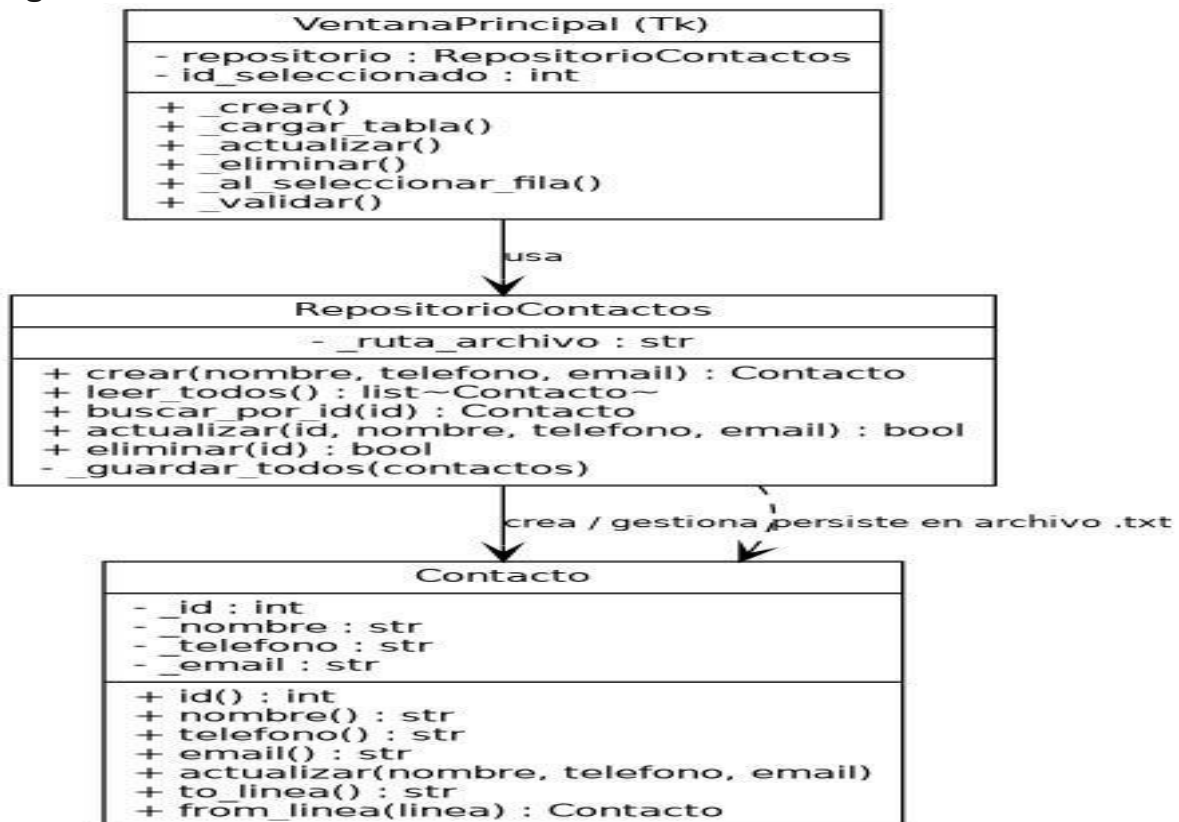
2. Arquitectura de clases

Contacto: clase modelo con atributos privados (`_id`, `_nombre`, `_telefono`, `_email`) expuestos mediante propiedades, y métodos para convertirse hacia/desde una línea de texto del archivo.

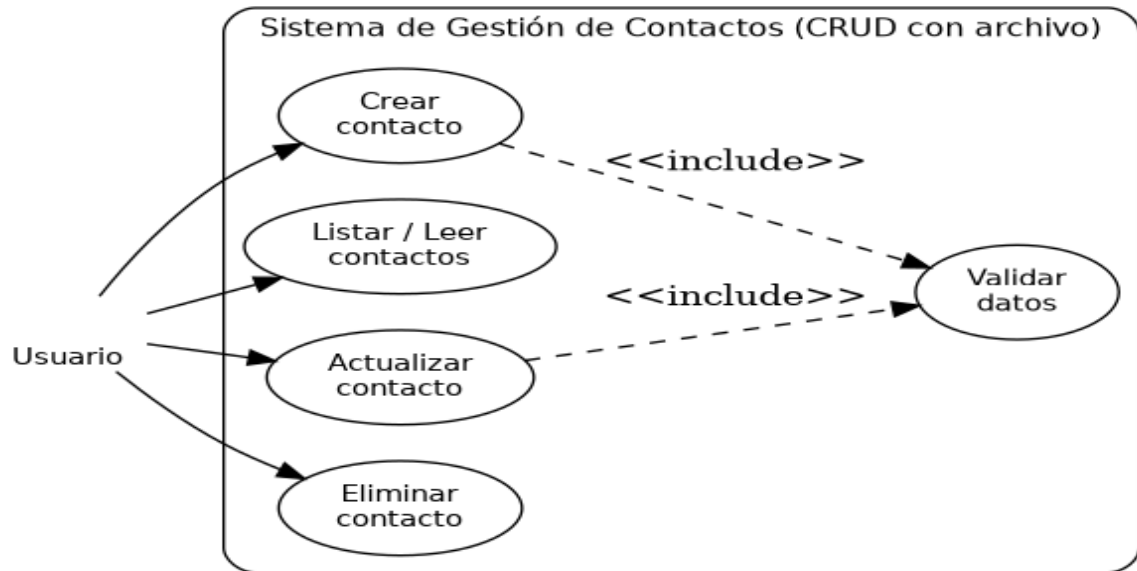
RepositorioContactos: encapsula todo el manejo del archivo (apertura, lectura, escritura) y expone los métodos `crear()`, `leer_todos()`, `actualizar()` y `eliminar()`.

VentanaPrincipal: interfaz gráfica construida con Tkinter que arma el formulario, la tabla (Treeview) y los botones CRUD, y delega toda la persistencia en el repositorio.

3. Diagrama de clases



4. Diagrama de casos de uso



5. Interfaz de usuario

Captura de pantalla real de la aplicación en ejecución (python app.py), mostrando el formulario, los botones Create / Read / Update / Delete y la tabla de contactos leída desde el archivo.

La interfaz de usuario de la aplicación 'Gestión de Contactos - CRUD con archivo' se muestra en una ventana con los botones de control estándar (minimizar, maximizar, cerrar). La interfaz está dividida en tres secciones principales:

- Datos del contacto:** Una sección superior que contiene tres campos de entrada de texto etiquetados como 'Nombre:', 'Teléfono:' y 'Email:'.
- Botones de acción:** Una fila de cinco botones situados debajo de los campos de entrada: 'Create', 'Read / Refrescar', 'Update', 'Delete' y 'Limpiar'.
- Contactos guardados en el archivo:** Una sección inferior que contiene una tabla con cuatro columnas: 'ID', 'Nombre', 'Teléfono' y 'Email'. La tabla está actualmente vacía.

6. Enlace del repositorio en GitHub

Repositorio: <https://github.com/juanjosevelezbustamante/Trabajos-de-POO/tree/329f0ddc34ba5091449d70fd7b129d51a4d42c82/proyecto>

7. Instrucciones de ejecución

```
cd proyecto/src
python app.py
```

Requisitos: Python 3.10+, Tkinter
(Linux: `sudo apt install python3-tk`)

8. Manejo de archivo (extracto de código)

```
Operaciones CRUD implementadas en RepositorioContactos: def
crear(self, nombre, telefono, email):    contactos = self.leer_todos()
nuevo_id = (max([c.id for c in contactos]) + 1) if contactos else 1
nuevo = Contacto(nuevo_id, nombre, telefono, email)    with
open(self._ruta_archivo, "a", encoding="utf-8") as archivo:
    archivo.write(nuevo.to_linea() + "\n")    return
nuevo

def leer_todos(self):    contactos = []    with
open(self._ruta_archivo, "r", encoding="utf-8") as archivo:
for linea in archivo:        if linea.strip():
contactos.append(Contacto.from_linea(linea))    return
contactos def actualizar(self, id_contacto, nombre,
telefono, email):    contactos = self.leer_todos()    for
c in contactos:        if c.id == id_contacto:
c.actualizar(nombre, telefono, email)
self._guardar_todos(contactos)    return True
return False

def eliminar(self, id_contacto):    contactos =
self.leer_todos()    nuevos = [c for c in contactos if
c.id != id_contacto]    if len(nuevos) == len(contactos):
return False
self._guardar_todos(nuevos)    return
True
```